

UNIVERSIDAD TECNOLÓGICA NACIONAL

Tecnicatura Universitaria en Programación — Modalidad a Distancia

Programación 3

Food Store

Sistema de Gestión de Pedidos de Comida

Trabajo Práctico Integrador (TPI)

Alumno: Nicolás Viruel

Materia: Programación 3

Metodología: Ágil / Scrum

Fecha de entrega: Junio de 2026

Informe TPI — Food Store

Índice

1. Introducción
 - 1.1. Objetivos del proyecto
 2. Marco Teórico
 - 2.1. Tecnologías del Backend
 - 2.2. Tecnologías del Frontend
 3. Decisiones Técnicas y Arquitectura
 - 3.1. Arquitectura general
 - 3.2. Arquitectura del Backend: arquitectura en capas
 - 3.3. API REST — Endpoints principales
 - 3.4. Arquitectura del Frontend
 - 3.5. Sistema de Autenticación y Autorización
 - 3.6. Funcionalidades por módulo
 - 3.7. Flujos de usuario principales
 - 3.8. Datos de prueba
 4. Dificultades y Soluciones
 5. Consideraciones de Seguridad (ámbito educativo)
 6. Conclusión
 7. Bibliografía / Webgrafía
- Anexo A — Cómo ejecutar el proyecto

1. Introducción

Food Store es una aplicación web *full stack* que implementa un *ecommerce* orientado a la venta de productos de un negocio de comidas. El sistema permite gestionar **categorías**, **productos** y **pedidos**, y cuenta con una gestión de usuarios basada en dos perfiles principales:

- **ADMIN:** accede al panel de administración para realizar operaciones CRUD sobre categorías y productos, y para gestionar el ciclo de vida de los pedidos.
- **USUARIO (cliente):** puede registrarse, iniciar sesión, navegar el catálogo, seleccionar productos, administrar un carrito persistente, confirmar compras y consultar el historial y el estado de sus pedidos.

El desarrollo se abordó como un proyecto *full stack* guiado por un *backlog* Scrum, donde las funcionalidades se organizaron en épicas e historias de usuario. Cada historia se transformó en pantallas concretas del *frontend* (según el rol) y en *endpoints* de la API REST del *backend*, respetando criterios de aceptación, validaciones y reglas de negocio.

Este documento describe las tecnologías empleadas (marco teórico), las decisiones técnicas y la arquitectura de la solución, las principales dificultades encontradas durante el desarrollo y sus soluciones, acompañado de la bibliografía utilizada como apoyo.

1.1. Objetivos del proyecto

Desarrollar una aplicación web *full stack* completa para la gestión de un negocio de comidas,

que permita:

- 1. **A los administradores:** gestionar categorías, productos y pedidos.
- 2. **A los clientes:** navegar productos, realizar compras y seguir sus pedidos.
- 3. **Carrito de compras:** funcional y con persistencia en `localStorage`.
- 4. **Integración completa:** conexión *frontend-backend* mediante una API REST.

2. Marco Teórico

En esta sección se describen brevemente las tecnologías utilizadas y los conceptos clave aplicados en el proyecto.

2.1. Tecnologías del Backend

Tecnología	Versión	Rol en el proyecto
Java	17 (LTS)	Lenguaje de programación del servidor.
Spring Boot	3.2.5	<i>Framework</i> para construir la aplicación y el servidor embebido.
Spring Web (MVC)	3.2.5	Exposición de controladores REST.
Spring Data JPA	3.2.5	Persistencia y repositorios sobre Hibernate.
Hibernate	(gestionado por Boot)	Implementación ORM (mapeo objeto-relacional).
Spring Validation	3.2.5	Validación declarativa de DTOs (Bean Validation).
Spring Security Crypto	—	Hashing de contraseñas con BCrypt .
springdoc-openapi	2.3.0	Documentación interactiva de la API (Swagger UI).
Lombok	—	Reducción de <i>boilerplate</i> (builders, accesores).
H2 Database	(embebida)	Base de datos en modo archivo, con fines educativos.
Gradle	<i>Wrapper</i>	Gestión de dependencias y construcción del proyecto.

Conceptos clave aplicados

- **API REST:** comunicación cliente-servidor basada en recursos (`/api/products` , `/api/orders` , etc.) y verbos HTTP (`GET` , `POST` , `PUT` , `PATCH` , `DELETE`).
- **ORM / JPA:** las clases del modelo se mapean a tablas mediante anotaciones (`@Entity` , `@Table` , `@ManyToOne` , `@OneToMany`), evitando escribir SQL manualmente.
- **Inyección de dependencias:** Spring administra el ciclo de vida de los componentes (`@Service` , `@RestController` , `@Component`) e inyecta colaboradores por constructor.
- **Patrón DTO (Data Transfer Object):** se desacopla el modelo de dominio de los datos expuestos por la API.
- **Transaccionalidad:** las operaciones que modifican varios registros se ejecutan dentro de una transacción (`@Transactional`).

2.2. Tecnologías del Frontend

Tecnología	Rol en el proyecto
TypeScript	Lenguaje tipado que compila a JavaScript; aporta seguridad de tipos.
Vite	Build tool y servidor de desarrollo, configurado como MPA (Multi-Page Application).
HTML5	Estructura de cada página.
CSS3	Estilos de la interfaz.
Fetch API	Cliente HTTP nativo del navegador para consumir la API REST.
localStorage	Persistencia de la sesión y del carrito en el navegador.

Conceptos clave aplicados

- **MPA (Multi-Page Application):** cada vista es una página HTML independiente con su propio *script* de entrada (login, registro, catálogo, detalle, carrito, pedidos, panel admin).
- **Tipado estático y *type guards*:** las respuestas de la API se validan en tiempo de ejecución antes de tratarlas como datos confiables.
- **Separación de responsabilidades:** capa de tipos (`types/`), utilidades y cliente HTTP (`utils/`), y vistas (`pages/`).

3. Decisiones Técnicas y Arquitectura

3.1. Arquitectura general

La solución sigue una arquitectura **cliente-servidor desacoplada**: un *frontend* MPA en TypeScript que se comunica, exclusivamente por HTTP/JSON, con un *backend* REST en Spring Boot. La base de datos H2 persiste los datos del lado del servidor.

El siguiente esquema resume la comunicación y el flujo interno de una petición:

Frontend (cliente)	Canal	Backend (servidor)	Persistencia
Vite MPA — TypeScript, HTML, CSS. Sesión y carrito en <code>localStorage</code> .	↔ HTTP / JSON (<code>/api/**</code>)	Spring Boot REST (Java 17). Flujo: Controller → Service → Repository → Model .	↔ Base de datos H2 (modo archivo)

3.2. Arquitectura del Backend: arquitectura en capas

El *backend* se organiza siguiendo una **arquitectura en capas** que separa responsabilidades. El paquete raíz es `com.tuuniversidad.foodstore`:

Capa / Paquete	Responsabilidad
model/	Entidades JPA del dominio (Usuario , Producto , Categoria , Pedido , DetallePedido) y enums .
repository/	Interfaces de Spring Data JPA para el acceso a datos.
service/impl/	Lógica de negocio, validaciones y <i>mappers</i> entidad ↔ DTO.
controller/	Controladores REST que exponen los <i>endpoints</i> /api/** .
dto/	Objetos de transferencia de datos (entrada y salida de la API).
exception/	Excepciones de dominio y manejador global de errores.
(raíz)	Configuración: CorsConfig , OpenApiConfig , DataLoader .

El flujo de una petición es: **Controller → Service → Repository → Base de datos**, y la respuesta se devuelve mapeada a un **DTO**, nunca exponiendo directamente la entidad.

Diseño del modelo de dominio (UML)

El modelo de datos respeta el diagrama UML de la consigna. Sus decisiones de diseño más relevantes son:

- **Clase base Base (@MappedSuperclass)**: todas las entidades heredan campos comunes:
 - id (clave primaria autogenerada),
 - eliminado (flag para **borrado lógico / soft delete**),
 - createdAt y updatedAt (auditoría automática vía @CreationTimestamp / @UpdateTimestamp),
 - version (control de **bloqueo optimista** con @Version).
- **Interfaz Calculable**: define el contrato calcularTotal() , implementado por Pedido . Es una aplicación directa de polimorfismo y del principio de segregación de interfaces.
- **Relaciones**:
 - Usuario 1—* Pedido (un usuario tiene muchos pedidos).
 - Pedido 1—* DetallePedido (composición, con cascade = ALL y orphanRemoval = true).
 - Categoria 1—* Producto , y Producto *—1 Categoria (relación bidireccional gestionada).
 - DetallePedido *—1 Producto .
- **Enumeraciones**: Rol (ADMIN , USUARIO), Estado (PENDIENTE , CONFIRMADO , TERMINADO , CANCELADO) y FormaPago (TARJETA , TRANSFERENCIA , EFECTIVO), persistidas como STRING para mayor legibilidad en la base de datos.

Decisión: uso de BigDecimal para valores monetarios

El precio de los productos se modeló con **BigDecimal** (precision = 12 , scale = 2) en lugar de double , evitando los errores de redondeo propios de la aritmética en punto flotante. El cálculo de subtotales (precio × cantidad) y del total del pedido (subtotales + costo de envío) se realiza con **BigDecimal** , garantizando precisión en los importes.

Decisión: patrón DTO + Mappers

Cada entidad cuenta con sus DTOs y un *mapper* dedicado en `service/impl/` (`UsuarioMapper` , `ProductoMapper` , `PedidoMapper` , `CategoriaMapper`). Esto permite:

- **No exponer la contraseña** ni datos internos en las respuestas (el `UsuarioDto` no incluye la contraseña).
- Tener DTOs específicos por operación: `...Create` , `...Edit` , `...Dto` (salida), y casos puntuales como `PedidoStatusUpdate` o `PedidoCreateRequest` .
- Desacoplar el contrato de la API de la estructura interna de la base de datos.

Decisión: borrado lógico (*soft delete*)

En lugar de eliminar físicamente los registros, las operaciones de borrado marcan `eliminado = true` (`eliminarLogico`). Las consultas filtran por `findByEliminadoFalse...` , preservando la integridad histórica (por ejemplo, un pedido no pierde la referencia a un producto que fue “eliminado”).

Decisión: manejo centralizado de excepciones

Mediante un `@RestControllerAdvice` (`GlobalExceptionHandler`) se centraliza el tratamiento de errores y se devuelve un cuerpo JSON homogéneo (`status` , `error` , `timestamp`):

Excepción	Código HTTP
<code>ResourceNotFoundException</code>	404 Not Found
<code>BadRequestException</code>	400 Bad Request
<code>UnauthorizedException</code>	401 Unauthorized
<code>MethodArgumentNotValidException</code> (validación)	400 Bad Request
<code>IllegalStateException</code> / <code>OptimisticLockException</code>	409 Conflict
Exception (genérica)	500 Internal Server Error

Esto evita repetir lógica de manejo de errores en cada controlador y ofrece respuestas consistentes al *frontend*.

Decisión: transaccionalidad en la creación de pedidos

El método `PedidoService.crear(...)` está anotado con `@Transactional` . Dentro de una única transacción:

1. valida los datos de entrada,
2. recupera el usuario y cada producto,
3. **reduce el stock** de cada producto,
4. arma el pedido con sus detalles y calcula el total,
5. persiste el pedido.

Si cualquier paso falla, la transacción se revierte por completo, evitando descontar stock sin haber generado el pedido.

3.3. API REST — Endpoints principales

La API se expone bajo el prefijo `/api` . Está documentada de forma interactiva con **Swagger UI** (`/swagger-ui.html`).

Recurso	Método	Endpoint	Descripción
Auth	POST	/api/auth/login	Inicio de sesión.
Auth	POST	/api/auth/register	Registro de cliente (auto-login).
Productos	GET	/api/products	Listar productos.
Productos	GET	/api/products/category/{id}	Productos por categoría.
Productos	GET	/api/products/{id}	Detalle de producto.
Productos	POST / PUT / DELETE	/api/products/{id}	CRUD (admin).
Categorías	GET / POST / PUT / DELETE	/api/categories/{id}	CRUD de categorías.
Pedidos	GET	/api/orders	Listar todos los pedidos (admin).
Pedidos	GET	/api/orders/user/{userId}	Pedidos de un usuario.
Pedidos	POST	/api/orders	Crear pedido (checkout).
Pedidos	PATCH	/api/orders/{id}/status	Actualizar estado del pedido.
Pedidos	PUT / DELETE	/api/orders/{id}	Editar / borrado lógico.
Usuarios	GET / POST / PUT / DELETE	/api/users/{id}	CRUD de usuarios.

3.4. Arquitectura del Frontend

El *frontend* es una **MPA** construida con Vite, donde cada vista es una página independiente organizada por rol y dominio funcional:

```

final-prog3/src/
├── types/           # IUser, IProducto, Rol, contratos de la API (api.ts)
├── utils/          # auth, cart, navegación, guards, formato, apiConfig
│   └── api/        # cliente HTTP (http.ts) y módulos por recurso
│                   # (authApi, productApi, categoryApi, orderApi)
├── pages/
│   ├── auth/       # login, registro
│   ├── store/      # home (catálogo), productDetail, cart
│   ├── client/     # orders (Mis Pedidos)
│   └── admin/      # dashboard, categories, products, orders

```

Decisiones del Frontend

- **Cliente HTTP centralizado (http.ts):** una función genérica `apiFetch<T>()` encapsula `fetch`, cabeceras JSON, el manejo de respuestas `204 No Content` y la conversión de errores HTTP en una clase `ApiError` con `status` y `message`. Toda la app maneja los errores de forma uniforme.
- **Tipado y type guards:** las respuestas de la API se validan antes de usarse. Por ejemplo, `getSession()` aplica el *guard* `isUser()` para garantizar que el objeto recuperado de `localStorage` tiene la forma esperada.
- **Gestión de sesión:** la sesión del usuario se guarda en `localStorage` (`auth.ts`), con funciones explícitas `getSession`, `setSession`, `clearSession`.
- **Carrito persistente:** el carrito se almacena en `localStorage` (`cart.ts`), persiste entre

sesiones y se limpia al confirmar el pedido.

- **Proxy de desarrollo:** en modo *dev*, Vite redirige las peticiones `/api` al *backend* (`localhost:8080`), evitando problemas de CORS durante el desarrollo.

3.5. Sistema de Autenticación y Autorización

El flujo de autenticación implementado es:

1. El usuario ingresa credenciales y el *frontend* envía `POST /api/auth/login` (o `/register`).
2. El *backend* valida las credenciales: busca el usuario por *email* y verifica la contraseña con **BCrypt** (`PasswordEncoder.matches`).
3. Si es correcto, devuelve los datos del usuario (sin contraseña).
4. El *frontend* guarda la sesión en `localStorage` y redirige según el rol.
5. Cada página protegida verifica la sesión; si no existe, redirige al *login*.

Roles y permisos:

- **ADMIN:** acceso completo al panel de administración (CRUD de categorías y productos, gestión de estados de pedidos).
- **USUARIO:** catálogo, búsqueda y filtros, carrito, *checkout* e historial de pedidos propios. **No** accede al panel de administración.

3.6. Funcionalidades por módulo

Módulo de Autenticación

- Login con validación de campos y manejo de errores.
- Registro de clientes (email válido, contraseña mínima de 6 caracteres) con auto-login.

Módulo Cliente — Store

- *Home* / catálogo con *sidebar* de categorías, búsqueda en tiempo real, filtros y ordenamiento (A-Z, Z-A, precio asc./desc.), *badge* del carrito y contador de productos.
- Detalle de producto con selector de cantidad validado contra el stock.
- Carrito con control de cantidades, resumen de costos, *modal* de *checkout* (teléfono requerido) y persistencia en `localStorage`.

Módulo Cliente — Mis Pedidos

- Historial de pedidos del usuario con tarjetas (número, fecha, estado con *badge* de color, resumen y total) y *modal* de detalle completo (desglose de costos y mensaje según estado).

Módulo de Administración

- *Dashboard* con estadísticas (totales de categorías, productos, pedidos y disponibles).
- CRUD de categorías y de productos mediante *modales* con validaciones.
- Gestión de pedidos: lista de todos los pedidos, filtro por estado y actualización de estado vía `PATCH /api/orders/{id}/status`.

3.7. Flujos de usuario principales

Flujo de compra (cliente): autenticación → navegar catálogo → ver detalle → agregar al carrito → revisar carrito → *checkout* → confirmar pedido (`POST /api/orders`) → ver “Mis Pedidos”.

Flujo de gestión de producto (admin): autenticación → panel → “Productos” → crear/editar (`POST / PUT /api/products`) → ver en tabla.

Flujo de gestión de pedido (admin): panel → “Pedidos” → ver detalle → cambiar estado → actualizar (`PATCH /api/orders/{id}/status`) → el cliente ve el cambio reflejado.

3.8. Datos de prueba

Al iniciar el *backend*, la clase `DataLoader` carga datos iniciales (solo si la base está vacía): **2 usuarios, 3 categorías, 10 productos y 3 pedidos** de ejemplo.

Rol	Email	Contraseña
Administrador	admin@admin.com	123456
Cliente	cliente@foodstore.com	usuario123

4. Dificultades y Soluciones

Durante el desarrollo se presentaron varios obstáculos técnicos. A continuación se resumen los más relevantes y cómo se resolvieron:

#	Dificultad	Solución aplicada
1	Precisión de importes. Usar <code>double</code> para precios producía errores de redondeo en subtotales y totales.	Se migró el precio a <code>BigDecimal</code> (<code>precision=12</code> , <code>scale=2</code>) y se realizaron los cálculos monetarios con <code>BigDecimal</code> .
2	Eliminación de registros con relaciones. Borrar físicamente un producto o pedido rompía referencias históricas.	Se implementó borrado lógico (<code>eliminado = true</code>) y consultas filtradas por <code>findByEliminadoFalse</code> .
3	CORS al consumir la API desde el frontend. El navegador bloqueaba las peticiones y, en particular, el método <code>PATCH</code> .	Se configuró <code>CorsConfig</code> habilitando los orígenes y métodos (incluido <code>PATCH</code>) y se usó el proxy <code>/api</code> de Vite en desarrollo.
4	Consistencia al crear un pedido. Era posible descontar stock sin que el pedido llegara a guardarse.	Se encapsuló la creación del pedido en un método <code>@Transactional</code> , garantizando atomicidad (todo o nada).
5	Errores inconsistentes en la API. Cada controlador devolvía errores con formatos distintos.	Se centralizó el manejo con un <code>@RestControllerAdvice</code> que produce un cuerpo de error homogéneo y los códigos HTTP correctos.
6	Exposición de datos sensibles. El modelo <code>Usuario</code> incluye la contraseña.	Se aplicó el patrón DTO : el <code>UsuarioDto</code> de salida no contiene la contraseña, que además se almacena <i>hasheada</i> con <code>BCrypt</code> .
7	Confiar en datos del <code>localStorage</code>. El objeto de sesión podía estar corrupto o manipulado.	Se añadieron <i>type guards</i> en TypeScript (<code>isUser</code>) para validar la forma de los datos antes de usarlos.
8	Validación de stock en el carrito. Permitir cantidades mayores al stock disponible.	Validación en el <i>frontend</i> (selector limitado al stock) y reducción de stock controlada en el <i>backend</i> al confirmar el pedido.

5. Consideraciones de Seguridad (ámbito educativo)

Este proyecto tiene **finés exclusivamente académicos** y, por lo tanto, no implementa un esquema de seguridad de nivel productivo:

- Las contraseñas se almacenan **hasheadas con BCrypt** en el *backend*.
- **No** se utilizan *tokens* JWT; la sesión se guarda en `localStorage` .
- La validación de rol es principalmente del lado del cliente, por lo que `localStorage` es fácilmente modificable.

Estas limitaciones son aceptables para el alcance del TPI, pero en un entorno real deberían reforzarse con autenticación basada en *tokens*, autorización validada en el servidor y protección de los *endpoints*.

6. Conclusión

El proyecto **Food Store** cumple con los objetivos planteados en la consigna: una aplicación *full stack* navegable y verificable que cubre los flujos principales de cliente y administrador. Se aplicaron buenas prácticas de ingeniería de software —arquitectura en capas, patrón DTO, manejo centralizado de excepciones, transaccionalidad, borrado lógico, auditoría y control de concurrencia optimista— y un *frontend* tipado con validación de datos y consumo ordenado de la API REST.

El desarrollo guiado por épicas e historias de usuario (Scrum) permitió transformar cada requisito en pantallas y *endpoints* concretos con criterios de aceptación claros, resultando en un producto coherente y mantenible.

7. Bibliografía / Webgrafía

- **Spring Boot** — Documentación oficial. <https://docs.spring.io/spring-boot/index.html>
- **Spring Framework** — Reference Documentation. <https://docs.spring.io/spring-framework/reference/>
- **Spring Data JPA** — Reference. <https://docs.spring.io/spring-data/jpa/reference/>
- **Hibernate ORM** — User Guide. <https://hibernate.org/orm/documentation/>
- **Jakarta Bean Validation** — Specification. <https://beanvalidation.org/>
- **springdoc-openapi** — Documentación. <https://springdoc.org/>
- **BCrypt / Spring Security Crypto** — <https://docs.spring.io/spring-security/reference/features/integrations/cryptography.html>
- **TypeScript** — Handbook. <https://www.typescriptlang.org/docs/>
- **Vite** — Guía oficial. <https://vitejs.dev/guide/>
- **MDN Web Docs** — Fetch API y localStorage. <https://developer.mozilla.org/>
- **H2 Database Engine** — <https://www.h2database.com/>
- **Gradle** — User Manual. <https://docs.gradle.org/>

Anexo A — Cómo ejecutar el proyecto

Requisitos previos: JDK 17 o superior y Node.js 18 o superior.

1. Backend (puerto 8080):

```
cd foodstore-backend
./gradlew bootRun      # En Windows: gradlew.bat bootRun
```

- API: <http://localhost:8080>
- Swagger UI: <http://localhost:8080/swagger-ui.html>

2. Frontend (puerto 5173):

```
cd final-prog3
npm install
npm run dev
```

Abrir: <http://localhost:5173/src/pages/auth/login/index.html>

La base de datos H2 funciona en modo archivo (`jdbc:h2:file:./data/pedidos_db`); los datos persisten en `foodstore-backend/data/`.